

对《模块间服务接口契约梳理流程》的 6 条核心改进建议

您好，仔细阅读了工程团队提供的这份《模块间服务接口契约梳理流程》PDF。整体框架非常有大厂风范，在“解耦”和“异步大文件传递”上抓住了核心痛点。

但站在量化投研体系的实战视角，这份规范在“数据安全”、“缓存算力”以及“特征多阶消费”的考量上还有几处关键缺失。建议向工程团队反馈以下 6 条核心改进意见，要求补充到流程规范中：

1. 缺失 PIT (防穿越) 审计契约

- 痛点**：规范的 Request 示例里只有 `input_uri` 和 `params`。在量化系统中，如果接口不带时间安全参数，下游极易读到未来数据（例如在回测 2023 年财报时读到了 2024 年的修订版），从而导致致命的未来函数泄露。
- 改进**：强制要求所有查数据的 Query API 必须带入 `knowledge_ts`（物理可见时间）字段，严禁只传 `trade_date`。服务方必须据此进行防穿越拦截。

2. Artifact 没有体现“多阶预处理视图”

- 痛点**：规范的第 6 步把产物定义得过于单一（一个路径完事）。但在量化流程中，一个因子往往拥有 Raw（原始值）、Winsorized（去极值）、Neutralized（中性化）等多阶视图，下游不同的模型（树模型 vs 神经网络）需要取用不同阶数的特征。
- 改进**：要求在 Artifact/Manifest 契约中增加 `view_stage`（处理阶数）的概念。必须能支持表达并返回“第几阶处理后的特征”，避免混淆。

3. Error 语义缺少“业务静默拦截”

- 痛点**：规范列出的都是工程级报错（如 `Config_Invalid`, `OOM` 等）。但在因子评估中，最常见的是“因为夏普太低或相关性太高被拒”。这在工程上不算代码 Error，不需要无限重试。
- 改进**：在错误语义契约里补充 `BUSINESS_REJECTED` 这种量化特有的业务状态码。明确说明此类情况属于正常拦截，不需要触发 Job 重试机制。

4. 缺失“因子血缘 (Lineage)”与“数据版本”的强制契约

- 痛点**：规范要求 Manifest 中记 `git_commit`（代码版本）。但在我们场景中，一个因子今天赚钱明天不赚钱，可能不是代码改了，而是上游的行情快照（Data Version）重刷了。仅记录代码版本无法追溯数据变化。
- 改进**：强制在 Manifest 里写入 `data_lineage` 字段，明确记录它是由哪个 `source_factor_id` 衍生来的，以及它当时读取的底层数据快照是哪个版本，为未来的“级联失效重算”提供依据。

5. 对“幂等性与缓存命中”定义模糊

- 痛点**：规范里只是轻描淡写地提了一句“幂等标识”。但在量化回测中，如果两个研究员提交了同一个因子（只是名字不同），或者昨天失败的任务今天重跑，系统如果不做强缓存，将白白烧掉海量算力。
- 改进**：在 Response 契约中强制要求返回 `cache_hit: boolean`。系统底层应把“表达式 Hash + 时间范围 + 预处理模板”当成唯一的 Key，如果算过，直接秒回历史 Manifest 路径并标明 `cache_hit=true`，坚决不起新任务。

6. 缺失防篡改的“强只读 (Immutability)”与鉴权审计

- 痛点**：金融系统最怕的不是代码写错，而是“事后偷偷改数据”。如果跑完回测，有人偷偷去硬盘里改了 Artifact 里的某个数字粉饰夏普率，规范里定义的 `output_visibility` 根本防不住这种底层篡改。
- 改进**：强制要求所有的 Artifact 文件落盘后，文件系统/对象存储权限锁定为**不可变只读 (Immutable)**。并且 Manifest 文件里必须自动写入产物文件的 `SHA256 Checksum`（校验和）。下游一读发现 Hash 对不上，立刻报致命错误，彻底断绝手工篡改数据的可能。